Paperplane: zero to mastery

Local document extraction with Docling, OpenAI, and Agnes

Table of Contents

1. Run the app

Double-click Paperplane.cmd. It installs uv, Python 3.12.10, locked dependencies, and Docling layout/table models before starting Streamlit at http://127.0.0.1:8551. Set the provider key documented in MODELS.md for the selected scan/image model.

2. Follow the code

1. Start at streamlit_app.py.
2. Follow paperplane.runtime.parse_document.
3. Read AgenticDocumentParser.parse in paperplane/parser.py.
4. Inspect validation and page classification in paperplane/ingest.py.
5. Follow native conversion in paperplane/docling_parser.py or vision work in

paperplane/pipeline.py.

6. Read the model catalog and provider boundaries in paperplane/model_catalog.py,

paperplane/openai_document.py, paperplane/gemini_document.py, paperplane/anthropic_document.py, and paperplane/agnes_document.py.

7. Read coordinate alignment in paperplane/grounding.py.
8. Inspect output assembly in paperplane/contracts.py.
9. Follow evidence creation in paperplane/annotated_pdf.py.

3. Understand the data flow

```
uploaded bytes
  -> validation and document inspection
  -> per-page native/scanned routing
  -> local Docling conversion OR selected cloud-model structured vision draft
  -> deterministic grounding and mode-driven verification
  -> merge in source reading order
  -> provider token aggregation and estimated model cost
  -> Markdown + hierarchical JSON + annotated evidence PDF
  -> Output/PDF/Markdown/JSON display and download
```

4. Understand the contract

Physical pages are one-based and use normalized top-left-origin boxes. JSON nodes carry half-open Unicode Markdown ranges. Office content without trustworthy geometry is semantic_only with null boxes. IDs are stable within one response only.

5. Understand the state boundary

Streamlit keeps widget values, uploaded bytes, the latest result, and the annotated PDF in one session. The parser receives bytes and returns validated Pydantic values without saving either. The cached Docling converter holds model resources only.

6. Change the app safely

Keep parser logic independent from Streamlit, validate input at the parser boundary, keep secrets and documents out of logs and source control, add focused tests, and update affected documentation with every code change.

7. Verify

```
uv run ruff check paperplane tests streamlit_app.py scripts
uv run ruff format --check paperplane tests streamlit_app.py scripts
uv run pyright
uv run pytest tests -q
```

Continue with the full study handbook, model catalog, architecture, and capabilities.